



FINAL REPORT

Egg Catcher Arcade Game Project

Course: OBJECT-ORIENTED PROGRAMMING (IT069IU)

GROUP: J

TEST ROOM: LA1.605

TERM: Final Examination (2025-2026) S2

Ho Chi Minh City, May 27th 2026

GROUP PROJECT MEMBERS LIST

No.	Full Name	Student ID	Role	Contribution
1	Nguyễn Võ Minh Huy	ITITWE24044	Team Leader / Project Manager	20%
2	Nguyễn Xuân Thành	ITITWE24069	Lead Backend Software Engineer	20%
3	Ngô Nhật Anh	ITITWE24031	UI/UX Component & Asset Designer	20%
4	Nguyễn Phúc Vương	ITITWE24074	Quality Assurance / Testing Engineer	20%
5	Vũ Trần Anh Quân	ITITWE24020	Core Graphics Engine Programmer	20%

TABLE OF CONTENTS

ABSTRACT

CHAPTER 1. INTRODUCTION & SYSTEM ENGINE TIMINGS

- 1.1 Project Overview and Technical Constraints
- 1.2 Core Gameplay Loop

CHAPTER 2. PROJECT ARCHITECTURE AND DIRECTORY MAPPING

- 2.1 Workspace Directory Layout
- 2.2 Game Entities & Behaviors

CHAPTER 3. CLASS DEFINITIONS & EXECUTION FLOWS

- 3.1 Core Class Descriptions
- 3.2 Tier-Based Execution Flow

CHAPTER 4. OBJECT-ORIENTED PROGRAMMING DESIGN ENHANCEMENTS

- 4.1 Strict Separation of Concerns
- 4.2 Observer Pattern via Custom Interfaces
- 4.3 Memory Optimization through Packed Sprite Sheets

CHAPTER 5. SYSTEM OPERATIONS & INTEGRATED BUILD MANUALS

- 5.1 Environment Prerequisites Verification
- 5.2 Multi-Platform Execution Scripts

CHAPTER 6. SYSTEM EVALUATION & TECHNICAL CONCLUSIONS

ABSTRACT

Developed completely using pure Java and Swing abstract component toolkits, Egg Catcher is an optimized 2D desktop arcade experience built entirely without external rendering engines or bloated cross-platform packages. The primary goal of the application is to highlight the practical benefits of foundational Object-Oriented Programming (OOP) architectures when building interactive graphical software subsystems.

By enforcing an absolute decoupling between backend arithmetic entities and standalone loop rendering controllers, the game engine easily achieves reliable, lightweight computation profiles. Key patterns include structural decoupling via automated system loops, decoupled multi-threaded layout architectures, dynamic configuration mapping via packed resource atlas sheets, and complete isolation from host machine font configurations through embedded custom bitmap typography calculations. The resulting product delivers robust runtime stability, minimal resource usage, and smooth execution across heterogeneous computing platforms.

CHAPTER 1. INTRODUCTION & GAMEPLAY MECHANICS

1.1 Project Overview and Technical Constraints

In interactive 2D graphics programming, delivering a responsive user experience requires an optimized, clean architecture, particularly within the frame refresh cycle. The Egg Catcher project provides a stable 2D desktop application built completely within the boundaries of the standard Java Development Kit (JDK). By avoiding heavy external engines, this project demonstrates how foundational programming practices, structured data management, and optimized asset handling can achieve smooth game execution even on low-spec hardware.

1.2 Iterative Core Gameplay Mechanics and Loop Flow

The game operates on a continuous loop that handles state updates and rendering at fixed frame intervals through four structural steps:

Step 1 (Asset Spawning): A chicken entity moves automatically back and forth along the top of the screen, continuously updating its position and dropping egg objects downwards at randomized time intervals.

Step 2 (Input Handling): The system uses a dedicated keyboard listener thread to catch player inputs (Left/Right arrow keys) instantly. This allows the player's basket to move smoothly across the screen without interrupting or blocking the main physics loop.

Step 3 (Collision Detection): During every loop cycle, the engine evaluates the mathematical intersection between the bounding boxes of the falling eggs and the player's basket coordinates.

Step 4 (State & Score Updates): Successfully catching an egg increments the player's score. Allowing an egg to drop past the bottom boundary decreases the player's lives. The game loop continuously monitors health values and safely triggers a Game Over state when lives reach zero.

CHAPTER 2. PROJECT ARCHITECTURE AND DIRECTORY MAPPING

2.1 Unified Workspace Directory Hierarchy Layout

The source codebase, assets layout, configurations, binary outputs, and shell launch definitions are cleanly managed within a strict workspace project directory structure:

eggcatcher/	[Unified Master Root Project Folder]
assets/	[Static Resource Bundles & Graphic Grid Maps]
BoxyBold.fnt	- Character Bounds Typography Mapping Layouts
BoxyBold.png	- Custom Font Texture Atlas Sheet Graphic
egg-catcher.atlas	- Sprite Location Offsets Reference Dictionary
egg-catcher.png	- Compressed Pack of Graphical Sub-Images
src/	[Encapsulated Pure Java Source Source Codes]
eggcatcher/	- System Execution Packages & Module Logic
bin/	[Target Bytecode Translation Storage Path]
eggcatcher/	- Transformed Executable Java Bytecode (.class)
compile-and-run.bat	- One-Click Automated Compilation Launch for Windows OS
compile-and-run.sh	- Shell Script Execution Environment for Linux/macOS
README.md	- Operations Technical Guide Documentation

2.2 Game Entities Description & Behavioral Details

To ensure high rendering speeds, the application uses distinct object blocks that decouple game behavior from underlying loop processing:

Entity Component	Operational Architecture details
Chicken Class	Handles autonomous horizontal movement along the top boundary. It dynamically controls the drop timing and scales up its speed based on the player's current score to increase difficulty .
Falling Object (Egg)	Accelerates downward using simulated gravity variables. It monitors its own position for collisions or out-of-bounds events and handles its own lifecycle.
Basket Class	Binds directly to user keyboard inputs, handling horizontal adjustments and maintaining the player's active collision box.

CHAPTER 3. CLASS DEFINITIONS & TECHNICAL ARCHITECTURE MAPS

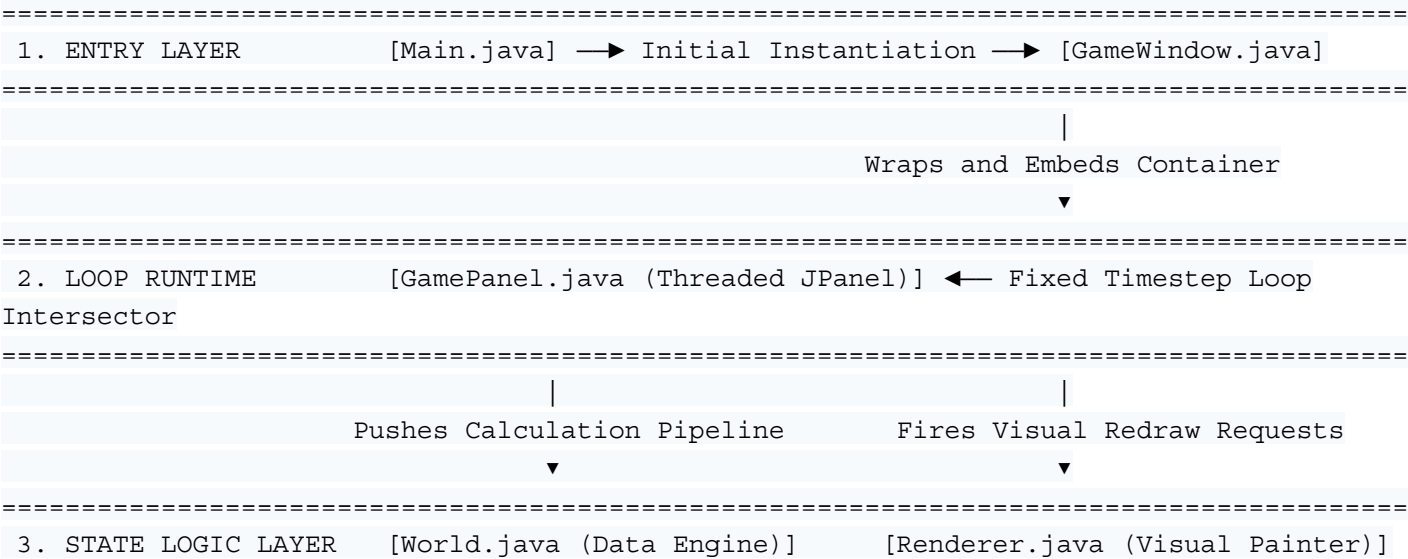
3.1 Core Class Descriptions and Compilations

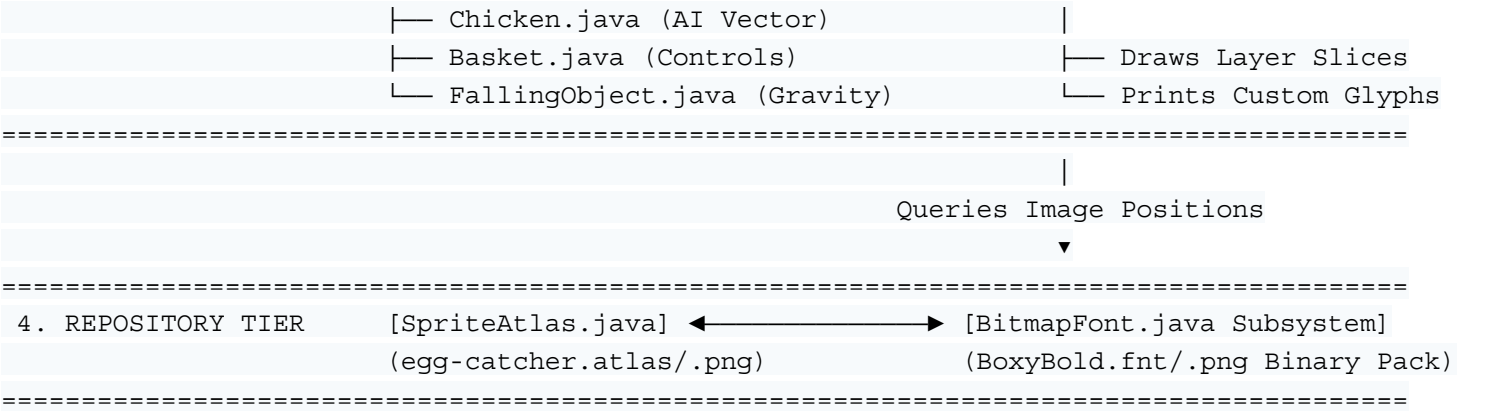
The system comprises exactly 10 internal production modules that divide runtime management tasks into specific, isolated scopes:

1. **Main.java:** Activates initialization loops and constructs standard screen deployment operations.
2. **GameWindow.java:** Handles system display frames, monitor alignments, boundaries, scaling constraints, and window parameters.
3. **GamePanel.java:** Employs standard thread structures to run the primary application timing loop, driving layout recalculations and view repaints at fixed intervals.
4. **World.java:** Tracks core transactional parameters, including user scores, health pools, level progression, and object coordinate structures.
5. **Chicken.java:** Controls vector updates, AI path logic, and item drops for the upper actor.
6. **FallingObject.java:** Calculates descending vertical motion curves using simulated gravitational acceleration.
7. **Basket.java:** Manages user coordinate positioning and maps arrow-key hardware interactions onto physical bounding boxes.
8. **Renderer.java:** Intercepts standard Graphics2D drawing streams to render backgrounds, actors, and texts cleanly onto the display component on every loop step.
9. **SpriteAtlas.java:** Slices combine image resources into reusable graphical elements without requiring hardcoded file paths.
10. **BitmapFont.java:** Renders typography maps pixel-by-pixel, isolating the user interface from host operating system text configurations.

3.2 Decoupled Tier-Based Structural Execution Flows

The runtime architecture isolates application processes across four distinct layers to prevent components from directly mutating unrelated execution steps:





CHAPTER 4. OBJECT-ORIENTED PROGRAMMING DESIGN ENHANCEMENTS

4.1 Strict Separation of Concerns

A common pitfall when writing interactive applications with simple 2D drawing setups is combining game state mathematics with render pipeline calls inside single classes. The Egg Catcher architecture enforces a clear boundary between data tracking (managed within `World`) and the frame refresh loop (handled within `Renderer`). This ensures updates to layout dimensions or color profiles do not alter underlying collision physics.

4.2 Advanced Observer Pattern via Custom Interfaces

To safely isolate physical positions from background system routines, the software implements a decoupled message publishing design via `World.Listener`. When events occur—such as point updates or health changes—the system broadcasts notifications out to all registered observers. This decoupling keeps entity code clean and isolated from the rest of the application.

4.3 Graphic Memory Optimization through Packed Texture Sheets

Loading multiple loose graphic assets from disk during execution loops can cause noticeable system stuttering. The application utilizes a centralized `SpriteAtlas` manager that pulls a single packed graphic grid sheet (`egg-catcher.png`) directly into memory at startup. Sub-images are extracted on the fly via coordinate offsets, avoiding file IO calls during active gameplay loops.

CHAPTER 5. SYSTEM OPERATIONS & INTEGRATED BUILD MANUALS

5.1 Environment Prerequisites Verification

The application relies entirely on core system frameworks and does not require third-party build software. Executing the code requires a standard Java Development Kit (JDK Version 8 or higher) environment. Confirm terminal access using the following verification command:

```
java -version
```

5.2 Automation Multi-Platform Execution Routines

The codebase includes cross-platform deployment utilities to compile source code files and launch execution routines in one step:

- **Windows Operating Systems:** Double-click the compile-and-run.bat batch command file, or execute it via standard consoles:

```
.\compile-and-run.bat
```

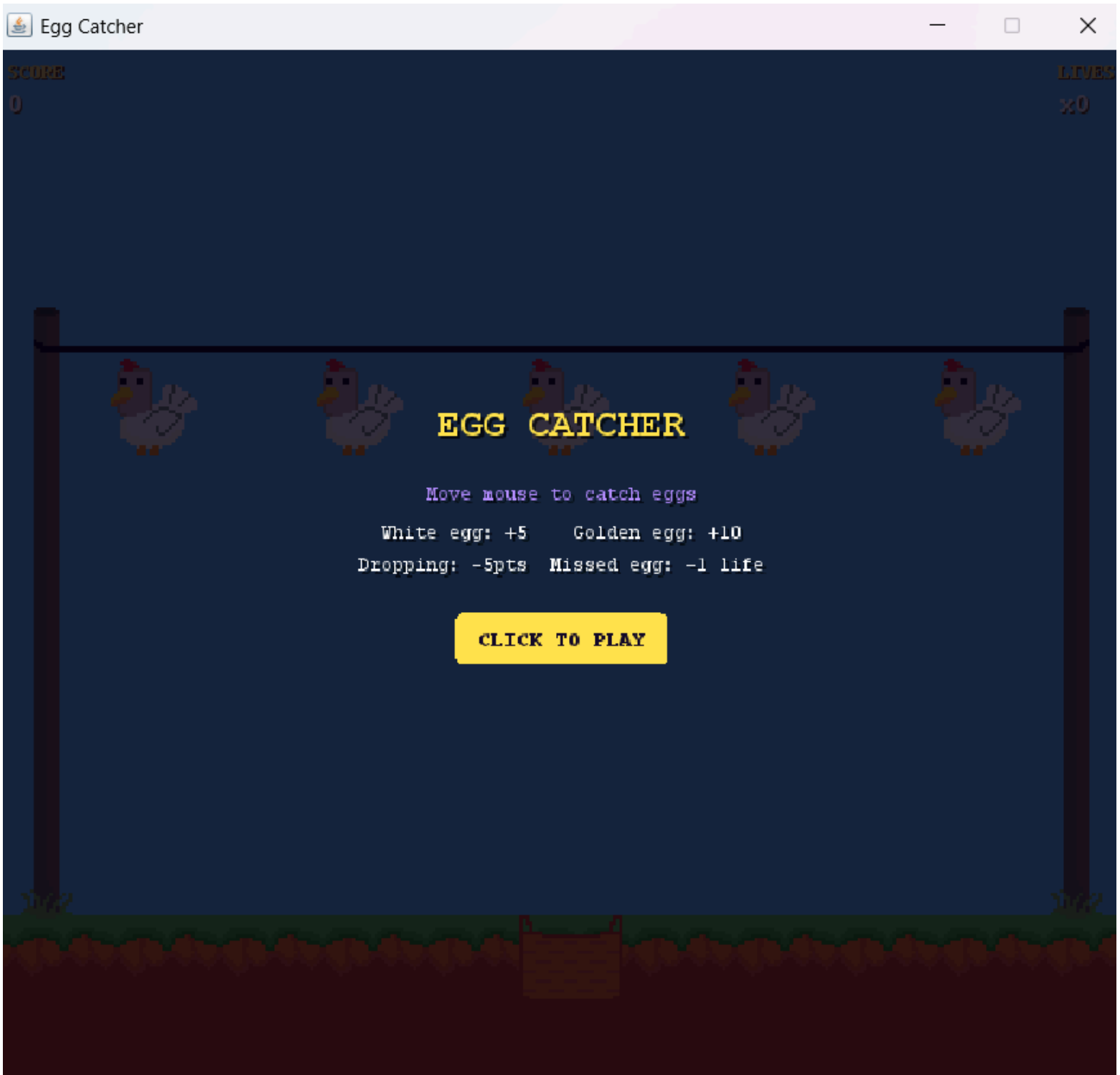
- **Unix / Linux / macOS Architectures:** Initialize terminal paths within the workspace root folder, grant executable rights, and trigger the shell deployment tool:

```
chmod +x compile-and-run.sh && ./compile-and-run.sh
```

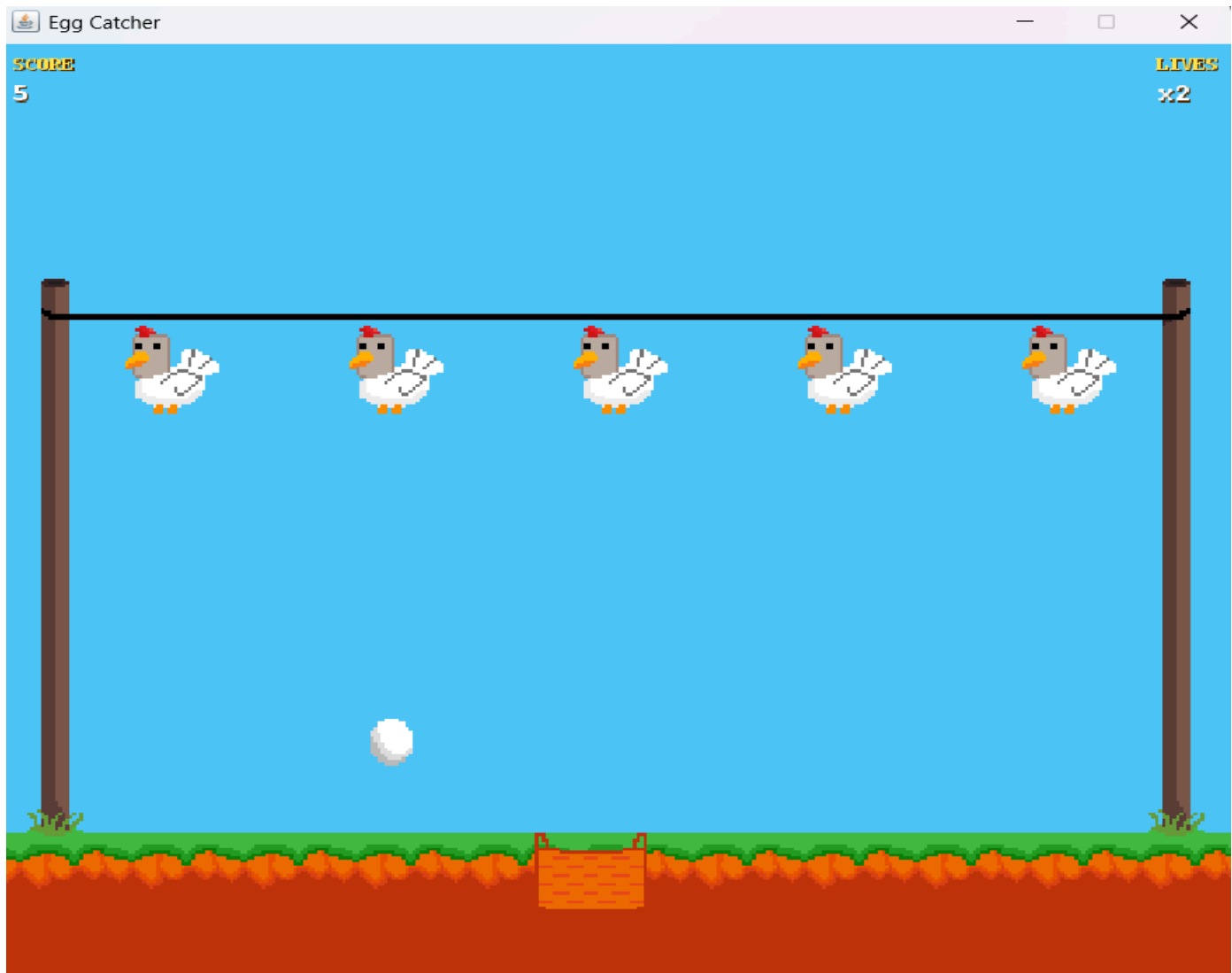
- **Interactive Controls Mapping:** After window activation, use the Left (←) and Right (→) arrow keys to shift the basket left and right beneath falling objects.

GAME SCREENS:

1. *Figure 1 : Main Gameplay Screen / Menu*



2. Figure 2 : The active gameplay screen, showing the pixel art chicken lineup and the descending egg entity.



3. Figure 3 : The Game Over terminal screen, displaying the final point state and state clean-up

SCORE

5

LIVES

x0

GAME OVER

Score: 5

CLICK TO PLAY AGAIN



CHAPTER 6. SYSTEM EVALUATION & TECHNICAL CONCLUSIONS

The **Egg Catcher Arcade Game Project** presents a clean, robust 2D framework that highlights foundational Object-Oriented Programming concepts. Spanning exactly 10 production classes working alongside custom text rendering logic, dynamic atlas slicing, and automated build scripts, the project architecture establishes a scalable structural baseline. This design ensures consistent runtime behavior and provides a strong foundation for future enhancements like multi-stage levels or advanced game design milestones.